

Build a Tiny GPT

Every piece, in about a hundred lines of code · video 2:19

This lesson puts the whole series into **one real program**: `tiny_gpt.py`, about a hundred lines of PyTorch. It reads a million characters of Shakespeare, builds a GPT with **818,241 parameters** from the pieces of episodes 2–10, trains it with the loop of episode 11 on an ordinary CPU, and watches it go from noise to something that looks like a play.

How to use this guide: master one concept before moving to the next.

1. Watch the whole video once, without stopping.
2. Then take the concepts **in order**. For each one: read the short explanation, look at the picture from the video (▶ is the time in the video), and answer every question in its *Check* without looking anything up.
3. Check your answers in the **Answers** section at the end (page 10).
4. **Pass mark: every question in the check correct.** If you missed one, rewatch that concept's video segment, reread the explanation, and redo only the questions you missed. Tick the concept off below, then move on.

It is strongly suggested that you **master each concept before starting the next one**. Each concept builds on the previous ones.

Concepts in this lesson

CONCEPT	VIDEO SEGMENT	QUESTIONS	MASTERED
1. Every character is a token	▶ 0:08-0:35	4	☐
2. Embeddings: token plus position	▶ 0:35-0:44	4	☐
3. Inside a block: attention and the MLP	▶ 0:44-1:03	4	☐
4. Logits out, and 818,241 parameters	▶ 1:03-1:21	4	☐
5. Training on an ordinary CPU	▶ 1:21-1:35	4	☐
6. Watching it learn	▶ 1:35-2:17	4	☐

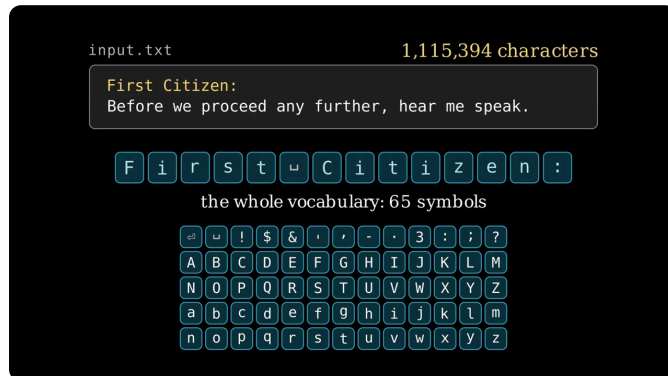
Before you start: this lesson assembles everything from episodes 1–11, so watch those first; episode 11 (training) matters most. The maths is in Foundations F05 (Exponentials and Logarithms), F10 (Slopes and Gradients), F12 (Neural Networks in Three Minutes), F13 (PyTorch and Autograd) and F14 (Train vs Validation Data). The hands-on exercises need Python with PyTorch.

CONCEPT 1 OF 6

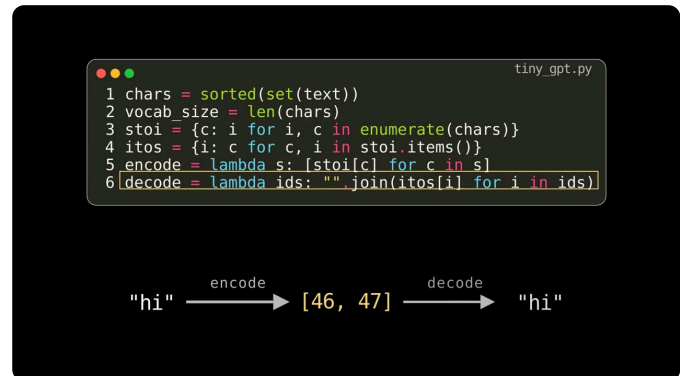
Every character is a token

▶ Video 0:08-0:35

Over eleven episodes we built every piece of a GPT. `tiny_gpt.py` puts them together and trains the result on an ordinary computer. The training text is **1,115,394 characters** of Shakespeare's plays. To keep things tiny, **every character is a token**, so the vocabulary is just the **65 different characters** in the file: newline, space, 10 punctuation marks, the digit 3, and the 52 capital and small letters.



▶ 0:28 1,115,394 characters of Shakespeare. The whole vocabulary: 65 symbols.



▶ 0:34 The tokenizer: characters to IDs (encode) and back (decode).

The tokenizer is **two small dictionaries**: `stoi` gives each character its position in the sorted list, and `itos` maps back. `encode("hi")` is `[46, 47]`, and `decode` turns that back into `hi`.

KEY IDEA

Character-level tokens: the vocabulary is the **65 characters** of the text, sorted, and a token's ID is its place in that list. Encoding and decoding are dictionary lookups.

Check 1: Every character is a token

Answer all 4 questions before looking at the answers. Pass mark: all correct.

1.1 MULTIPLE CHOICE

Why is the tiny GPT's vocabulary only 65 tokens?

- A. The model cannot handle more than 65 numbers
- B. Each token is one character, and the text uses only 65 different characters
- C. Rare words were removed from the text
- D. 65 is the length of the context

1.2 TRUE OR FALSE

"With this tokenizer, the word *the* is a single token."

True ☐ False ☐

1.3 CALCULATION

How many tokens is the text *"First Citizen:"*?

1.4 HANDS-ON CODE

Try it yourself. This code rebuilds the real 65-character vocabulary of `tiny_gpt.py` (the same list `sorted(set(text))` finds in the Shakespeare file) and its tokenizer. (a) Which IDs do the newline and the space get, and where do the capital and the small letters start? (b) Work out `encode("Hi!")` by hand, then check. (c) What is `decode([39, 1, 41, 39, 58])`? (d) What happens with `encode("4 cats")`, and why?

```
import string

chars = sorted("\n !$&',-.:;?" + string.ascii_uppercase + string.ascii_lowercase)
vocab_size = len(chars)
stoi = {c: i for i, c in enumerate(chars)}
itos = {i: c for c, i in stoi.items()}
encode = lambda s: [stoi[c] for c in s]
decode = lambda ids: "".join(itos[i] for i in ids)

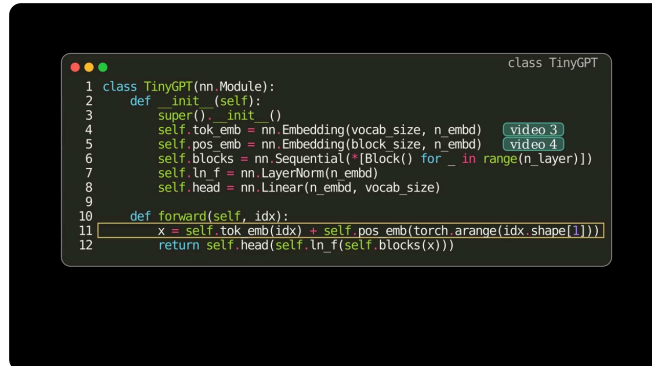
print(vocab_size, encode("hi"), decode(encode("hi")))      # 65 [46, 47] hi
```

CONCEPT 2 OF 6

Embeddings: token plus position

▶ Video 0:35-0:44

The model is the class `TinyGPT`. It starts with two **embedding tables**, as in episodes 3 and 4: one row of learned numbers per **token** and one per **position**, **added** together in `forward`.



▶ 0:43 class `TinyGPT`: a token table (video 3) and a position table (video 4).

```
self.tok_emb = nn.Embedding(vocab_size, n_embd)      # video 3
self.pos_emb = nn.Embedding(block_size, n_embd)      # video 4 (learned, like GPT-2)
x = self.tok_emb(idx) + self.pos_emb(torch.arange(idx.shape[1]))
```

So every character becomes **128 numbers** (`n_embd`) saying *which* character and *where*. The position table has only `block_size = 64` rows: the model reads at most 64 characters at a time.

KEY IDEA

x = token embedding + position embedding: one 128-number vector per character. The position table's 64 rows set the context length.

Check 2: Embeddings: token plus position

Answer all 4 questions before looking at the answers. Pass mark: all correct.

2.1 CALCULATION

How many learned numbers are in the two tables together?

.....

2.2 SHORT ANSWER

Input `hi = [46, 47]`. Which two rows are added to make the vector for `i`?

.....

2.3 CALCULATION

A batch is 32 sequences of 64 characters. What is the shape of `x` after the embeddings?

.....

2.4 TRUE OR FALSE

"The tiny GPT can read a context of any length, since positions are added, not stored."

True ☐ False ☐

CONCEPT 3 OF 6

Inside a block: attention and the MLP

▶ Video 0:44-1:03

Attention (episodes 5-7) makes queries, keys and values with **one linear layer** ($128 \rightarrow 3 \times 128 = 384$ numbers, split three ways), shared out over **4 heads** of 32 numbers. The scores $q \cdot k$ are divided by $\sqrt{32}$; the **causal mask** sets scores for later positions to $-\infty$, so after the **softmax** they get weight 0: no peeking ahead.

The **MLP** (episode 8) expands each vector to 512 numbers ($4 \times$), applies **GELU**, and projects back to 128. The **block** (episode 9) adds both onto the residual stream, each after a **layer norm**: $x = x + \text{self.attn}(\text{self.ln1}(x))$, then $x = x + \text{self.mlp}(\text{self.ln2}(x))$. Four blocks are stacked.

```

class MLP(nn.Module):
    def __init__(self):
        super().__init__()
        self.up = nn.Linear(n_embd, 4 * n_embd)
        self.down = nn.Linear(4 * n_embd, n_embd)

    def forward(self, x):
        return self.down(F.gelu(self.up(x)))

```

128 → up → $4 \times 128 = 512$ → GELU → down → 128

▶ 0:56 The MLP: $128 \rightarrow 4 \times 128 = 512 \rightarrow 128$.

```

class Block(nn.Module):
    def __init__(self):
        super().__init__()
        self.ln1, self.attn = nn.LayerNorm(n_embd), Attention()
        self.ln2, self.mlp = nn.LayerNorm(n_embd), MLP()

    def forward(self, x):
        x = x + self.attn(self.ln1(x)) # tokens talk
        x = x + self.mlp(self.ln2(x)) # each token thinks
        return x

```

```

1 block size, batch size = 64, 32
2 n_embd, n_head, n_layer = 128, 4, 4

```

Block
Block
Block
Block × 4

▶ 1:02 class Block, and 4 blocks stacked.

KEY IDEA

A block = **attention** (tokens talk: 4 heads, causal mask) + **MLP** (each token thinks: $128 \rightarrow 512 \rightarrow 128$), each added to the residual stream. The tiny GPT stacks 4.

Check 3: Inside a block: attention and the MLP

Answer all 4 questions before looking at the answers. Pass mark: all correct.

3.1 CALCULATION

$n_embd = 128$, $n_head = 4$. (a) How many numbers per head? (b) What are the scores divided by?

3.2 MULTIPLE CHOICE

What does `att.masked_fill(~self.mask[:T, :T], float("-inf"))` achieve?

- A. It removes padding characters
- B. No position can attend to later ones, so it cannot see what it must predict
- C. It keeps only the highest score in each row
- D. It switches off some heads at random

3.3 CALCULATION

With $n_embd = 256$ and 4 heads, how wide is the MLP's hidden layer, and how big is each head?

3.4 TRUE OR FALSE

"In each block, the attention output replaces `x`."

True ☐ False ☐

CONCEPT 4 OF 6

Logits out, and 818,241 parameters

▶ Video 1:03-1:21

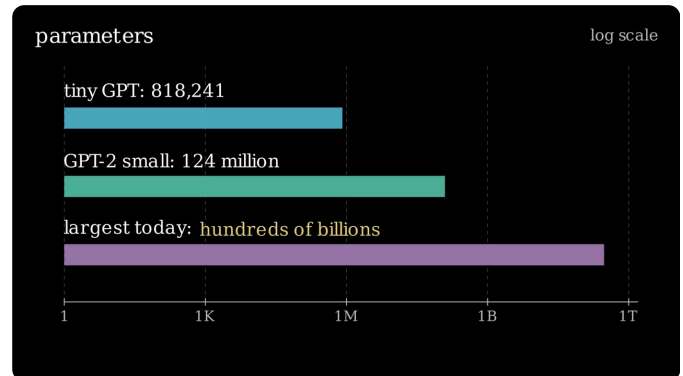
After the blocks come one last **layer norm** (`ln_f`) and a **linear layer** (`head`) that turns each 128-number vector into **65 logits**, one per character. Softmax turns them into next-character probabilities (episode 10).

```
class TinyGPT(nn.Module):
    def __init__(self):
        super().__init__()
        self.tok_emb = nn.Embedding(vocab_size, n_embd)
        self.pos_emb = nn.Embedding(block_size, n_embd)
        self.blocks = nn.Sequential(*[Block() for _ in range(n_layer)])
        self.ln_f = nn.LayerNorm(n_embd)
        self.head = nn.Linear(n_embd, vocab_size)

    def forward(self, idx):
        x = self.tok_emb(idx) + self.pos_emb(torch.arange(idx.shape[1]))
        return self.head(self.ln_f(self.blocks(x)))
```

→ 65 logits, one per character

▶ 1:10 One last layer norm, then a linear layer: 65 logits, one per character.



▶ 1:21 Parameters, on a log scale: from 818,241 to hundreds of billions.

That is the whole model: **818,241 parameters**. GPT-2 small has 124 million, and the largest models today have hundreds of billions. You can count them yourself: `nn.Linear(a, b)` has $a \times b$ weights plus b biases, `nn.LayerNorm(128)` has $128 + 128$, and `nn.Embedding(n, 128)` has $n \times 128$.

KEY IDEA

Output: `ln_f + head`, 128 → **65 logits** per position. In total **818,241 parameters**, almost all of them inside the 4 blocks.

Check 4: Logits out, and 818,241 parameters

Answer all 4 questions before looking at the answers. Pass mark: all correct.

4.1 CALCULATION

How many parameters does `head = nn.Linear(128, 65)` have?

4.2 MULTIPLE CHOICE

For each position, what comes out of the model?

- A. The ID of the next character
- B. 65 logits, one per character; softmax turns them into probabilities
- C. A vector of 128 numbers
- D. 64 probabilities, one per position

4.3 CALCULATION

About how many times more parameters does GPT-2 small (124 million) have?

4.4 HANDS-ON CODE

Try it yourself. Count the parameters by hand, then let PyTorch check. The code builds the layers of one block with the real sizes. (a) Before running, work out `count(qkv)`. (b) Predict the second number printed: one whole block. (c) Add up the model: 4 blocks, the two embedding tables (concept 2), the final layer norm and the head (4.1). Do you get 818,241? What share is in the blocks?

```
import torch.nn as nn

n_embd, n_head, n_layer = 128, 4, 4
count = lambda m: sum(p.numel() for p in m.parameters())

qkv = nn.Linear(n_embd, 3 * n_embd)      # Attention.qkv
proj = nn.Linear(n_embd, n_embd)         # Attention.proj
up = nn.Linear(n_embd, 4 * n_embd)       # MLP.up
down = nn.Linear(4 * n_embd, n_embd)     # MLP.down
ln = nn.LayerNorm(n_embd)               # a block has two: ln1, ln2

block = count(qkv) + count(proj) + count(up) + count(down) + 2 * count(ln)
print(count(qkv), block)
```

CONCEPT 5 OF 6

Training on an ordinary CPU

▶ Video 1:21-1:35

Training is the loop of episode 11, line for line: each step grabs **32 random chunks of 64 characters**, predicts **every next character**, computes the cross-entropy loss, backpropagates and steps with **AdamW**, a variant of Adam. The targets `y` are the same chunks one character further along:

```
tiny_gpt.py
1 opt = torch.optim.AdamW(model.parameters(), lr=lr)
2 ...
3 for step in range(max_steps + 1):
4     ...
5     x, y = get_batch(train_data)
6     logits = model(x) # forward pass
7     loss = F.cross_entropy(logits.view(-1, vocab_size), y.view(-1))
8     opt.zero_grad()
9     loss.backward() # backpropagation
10    opt.step() # nudge every weight
```

5,000 steps · 45 min on an 8-core CPU

▶ 1:35 The loop of episode 11, in `tiny_gpt.py`.

```
def get_batch(d):
    ix = torch.randint(len(d) - block_size - 1, (batch_size,))
    x = torch.stack([d[i:i + block_size] for i in ix])
    y = torch.stack([d[i + 1:i + block_size + 1] for i in ix]) # the next character
    return x, y
```

KEY IDEA

Each step: 32 chunks × 64 characters, **every position predicts its next character**. The first 90 % of the text is for training; **5,000 steps took about 45 minutes** on an ordinary 8-core CPU.

Check 5: Training on an ordinary CPU

Answer all 4 questions before looking at the answers. Pass mark: all correct.

5.1 CALCULATION

How many next-character predictions does one step score? About how many in 5,000 steps?

.....

5.2 SHORT ANSWER

With `block_size = 4`, a chunk starts at the *F* of “*First Citizen:*”. What are `x` and `y`?

.....

5.3 CALCULATION

`logits` is $32 \times 64 \times 65$. What shape is `logits.view(-1, vocab_size)`?

.....

5.4 CALCULATION

Of the 1,115,394 characters, the first 90 % are for training. How many does the model never train on?

.....

CONCEPT 6 OF 6

Watching it learn

▶ Video 1:35-2:17

The program prints samples as it trains (temperature 0.8, episode 10). **Step 0** (validation loss 4.41): pure noise, like `RYcb!FkBYGF`. **Step 250** (2.20): common letters, spaces in the right places, short lines, but made-up words such as *thamfors*. **Step 5,000** (1.59): real words, character names such as GLOUCESTER, and the shape of a play.

step 250 · val loss 2.20 sample · temperature 0.8

At 'T mat cos: lord, latt eay Saser hemour
Reabe my *thamfors* be make *hakeate* you shis a che plit?

That ance Drald at the ther the to at fou
Agrarve and lout thand ss not *asivrisked*; bats laghave;
O the put ing corn tond the wis crouge as you! than
Bu this some tous maId ardeck, down wongnd ingakeary,

common letters ✓
spaces ✓
short lines ✓
made-up words

▶ 1:49 Step 250 (validation loss 2.20): made-up words.

step 5000 · val loss 1.59 sample · temperature 0.8

At 'Timpercommand, know't, and says he will take my stamp?

GLOUCESTER:
Then shall we are 'twere?

Lord:
Yew that Henry's forgivery to the need and many
Statury highs nothing risonmentatiols, and they
droubt by my father's humane is
To asky in the friends with anoise for Aufidius;
My wong'd in a tardy,
Romes lies and loves thee sort and high rest.

CLARENCE:
Where is no strange to some to requit t

real words ✓
character names ✓
the shape of a play

learned only by
predicting the
next character

▶ 1:59 Step 5,000 (1.59): real words and names.

It isn't Shakespeare, but it learned all this from nothing but **predicting the next character**. Real LLMs use the **same recipe** with far more data, layers and compute; the two bonus episodes show how they generate text quickly and how a raw GPT becomes a chatbot.

KEY IDEA

Noise → common letters and spaces → words and structure, learned only from next-character prediction. Real LLMs: **the same recipe, much bigger**.

Check 6: Watching it learn

Answer all 4 questions before looking at the answers. Pass mark: all correct.

6.1 PUT IN ORDER

Put these real samples in training order: (A) `Reabe my thamfors be make hakeate` · (B) `GLOUCESTER: Then shall we are 'twere?` · (C) `RYcb!FkBYGF`.

6.2 CALCULATION

Which p, given to the correct character every time, matches the loss at step 0 (4.41) and at step 5,000 (1.59)?

6.3 SHORT ANSWER

At step 250 the model writes *thamfors* and *hakeate*. What has it learned, and what not yet?

6.4 TRUE OR FALSE

“Real LLMs are trained with a completely different recipe from the tiny GPT.”

True ☐ False ☐

ANSWERS

Answers and explanations

Mark each answer right or wrong. If a concept has any wrong answer, rewatch its segment (▶) and redo the questions you missed before moving on.

1. Every character is a token • rewatch ▶ 0:08-0:35

1.1 B. `sorted(set(text))` collects the distinct characters of the file.

1.2 False. Every character is a token: *t*, *h*, *e* are three. (The GPT-2 tokenizer of episode 2 makes *the* one token.)

1.3 14. One per character, and the space counts: $5 + 1 + 8$.

1.4 (a) 0 and 1; A is 13, a is 39 • (b) [20, 47, 2] • (c) "a cat" • (d) KeyError: '4' (a) Sorting by character code puts newline and space first, then the punctuation and the 3 (IDs 2-12), then A-Z (13-38) and a-z (39-64). (b) $H = 13 + 7 = 20$, $i = 39 + 8 = 47$, $! = 2$. (c) $39 = a$, $1 = \text{space}$, $41 = c$, $39 = a$, $58 = t$. (d) The only digit in the Shakespeare text is 3, so 4 has no ID: the model can only read and write the 65 characters it was trained on.

2. Embeddings: token plus position • rewatch ▶ 0:35-0:44

2.1 16,512. Token table $65 \times 128 = 8,320$; position table $64 \times 128 = 8,192$.

2.2 Row 47 of the token table + row 1 of the position table. *i* is token 47 and sits at position 1 (positions count from 0).

2.3 $32 \times 64 \times 128$. Each of the 32×64 characters becomes a vector of `n_embd` = 128 numbers.

2.4 False. `pos_emb` has only 64 rows, so position 64 has no row (PyTorch raises an error). That is why `generate` feeds only the last 64 characters: `idx[:, -block_size:]`.

3. Inside a block: attention and the MLP • rewatch ▶ 0:44-1:03

3.1 (a) 32. (b) $\sqrt{32} \approx 5.66$. `C // n_head` = $128 // 4 = 32$, and the code divides by `k.size(-1) ** 0.5` = $\sqrt{32}$.

3.2 B. $-\infty$ becomes 0 after the softmax. Without the mask, training would let the model read the answer.

3.3 1,024 and 64. $4 \times 256 = 1,024$ and $256 / 4 = 64$.

3.4 False. It is *added*: `x = x + self.attn(self.ln1(x))`. That running sum is the residual stream.

4. Logits out, and 818,241 parameters • rewatch ▶ 1:03-1:21

4.1 8,385. $128 \times 65 = 8,320$ weights plus 65 biases.

4.2 B. `head` maps 128 numbers to `vocab_size` = 65 scores. Picking a character happens later, in `generate`.

4.3 About 150 times. $124,000,000 / 818,241 \approx 152$: a bit more than two steps of $\times 10$ on the log scale.

4.4 (a) 49,536 • (b) 198,272 • (c) $4 \times 198,272 + 8,320 + 8,192 + 256 + 8,385 = 818,241$; about 97 % in the blocks (a) $128 \times 384 = 49,152$ weights + 384 biases. (b) qkv 49,536 + proj 16,512 + up 66,048 + down 65,664 + 2×256 . (c) The blocks hold 793,088 of the 818,241. The causal mask is not counted: it is a fixed buffer (`register_buffer`), not a learned parameter.

5. Training on an ordinary CPU · rewatch ► 1:21-1:35

5.1 2,048 per step; about 10 million in all. $32 \text{ chunks} \times 64 \text{ positions} = 2,048$, and $2,048 \times 5,000 = 10,240,000$.

5.2 $x = \text{"Firs"}$, $y = \text{"irst"}$. y starts one character later: each character of y is the one that follows the matching character of x .

5.3 $2,048 \times 65$. All positions of all chunks in one list: one row of 65 logits per prediction, and the loss is the average over the 2,048 rows.

5.4 111,540. `int(0.9 × 1,115,394)` = 1,003,854 for training; the other 111,540 are the validation text.

6. Watching it learn · rewatch ► 1:35-2:17

6.1 $C \rightarrow A \rightarrow B$. Noise at step 0, made-up words at step 250, real words and a character name at step 5,000.

6.2 About 0.012 (1 in 82) at the start, 0.20 (1 in 5) at the end. $\text{loss} = -\ln p$, so $p = e^{-\text{loss}}$: $e^{-4.41} \approx 0.012$ and $e^{-1.59} \approx 0.20$.

6.3 Common letters, spaces, short lines; not yet real words. The easy statistics of the text come first; words and names need more training.

6.4 False. Same recipe: next-token prediction, the same kind of blocks, the same training loop. Just far more data, layers and compute.